12.Linkage

La modularizzazione è quel metodo che organizza grandi programmi in piccole parti , le piccole parti hanno un interfaccia ben definita verso i moduli client che specifica come vengono resi disponibili i servizi forniti da questo modulo
Questa ha vari benefici specialmente in larghezza e per la complessi dei programmi

- i moduli possono esser utilizzati in vari progetti
- veloce ri-compilazione
- debbuging facile

Come lavora il compilatore in C

Questo svolge le operazioni su la unita di traduzione e consiste in :

- un file sorgente
- tutti i restanti file vengono definiti includendo tutte le direttive `#include`

Se nel caso il compilatore non abbia trovato nessun errore , questo genererà un file oggetto contenendo il corrispettivo codice macchina, spesso identificato con il suffisso del file `.o` o `.obj`
Questi vengono chiamati anche moduli

Come lavora il linker

Il compilatore successivamente invoca il linker che , combinerà gli oggetti , tutte le librerie usate nei file eseguibili
L'Header(.h) contiene solamente le macro definizioni , i tipi definiti e le funzioni dichiarate che il client program può vedere e usare
Nei file tipo `name.c` dobbiamo includere il file `name.h` , includendolo il compilatore prendere tutte le costanti tipi e variabili necessarie per l'utilizzo

Dichiarazione vs definizione

- Dichiarazione = annuncia al compilatore che esiste un identificatore (nome e type) ma non *alloca in memoria* , es:
  - extern int a , senza inizializzazione ; oppure void fun();
- Definizione = oltre a dichiarare , alloca la memoria per l'oggetto per la var. o fornisce il codice per la funzione
  - int a = 10 , o solo int a ; oppure void fun(){....}
- regola d'oro = possiamo avere molte dichiarazione , ma una sola ***definizione*** in tutto il programma.

Storage class specifiers

Questa in una dichiarazione modifica il linkage di un identificatore definito e lo spazio di durata del corrispettivo oggetto.

Una frequente fonte di confusione riguardo al linguaggio C è il fatto che il *linkage*, che è una proprietà degli identificatori, e la *storage duration*, che è una proprietà degli oggetti, siano entrambi influenzati nelle dichiarazioni dallo stesso insieme di parole chiave: gli *storage class specifiers*.

Storage class

- Gli identificatore di una funzione possono essere accompagnati solamente dallo specificatore di classe di memorizzazione `extern o staic`
- i parametri di una funzione posson esser accompagnati solo dallo specificatore della classe di memorizzazione `register`
- Gli identificatori di variabili possono avere `auto, register, static, extern.`

Linkage

il Linkage di un identificatore definisce che questo può esser usato o meno in un altrà unita di traduzione
È una sorta di ambito (scope) tra file differenti.

Un identificatore di funzione può avere:

- Connessione interna (internal linkage)
- Connessione esterna (external linkage)

Un identificatore di variabile può avere:

- Connessione interna (internal linkage)
- Connessione esterna (external linkage)
- Nessuna connessione (no linkage)

Linkage static (interno): usato su una funzione o variabile globale , la rende privata per quel file *#.c* , il linker non lo vede , quindi possiamo avere variabili con lo stesso nome in file diversi senza conflitto

Linkage extern(esterno): usato per condividere variabili globali su file diversi

Per riassumere

- **Un identificatore con connessione esterna (external linkage)** rappresenta la stessa funzione o oggetto in tutto il programma. Il compilatore presenta tali identificatori al linker, che li risolve con altre occorrenze in altre unità di traduzione e librerie.
- **Un identificatore con connessione interna (internal linkage)** rappresenta lo stesso oggetto o funzione all'interno di una determinata unità di traduzione. L'identificatore non viene presentato al linker. Di conseguenza, non è possibile utilizzare l'identificatore in un'altra unità di traduzione per riferirsi allo stesso oggetto o funzione.
- **Nessuna connessione (No linkage):**
  - Parametri di funzione.
  - Identificatori di oggetti dichiarati all'interno di una funzione e senza lo specificatore di classe di archiviazione `extern` .